Plexus Atlas — CompuCell3D

The second repository, and the first test of whether any of this transfers

1 August 2026

The twin of `atlas_jax_morph/atlas_note.pdf`. That one asked whether Plexus can say what one framework says. This one asks whether the *procedure* is a procedure.

---

1. Why a second repository, and why this one

One repository cannot show saturation. The measurement `plexus2.tex` App. E.1 asks for — *does decomposing a broad collection of frameworks converge on a compact reusable vocabulary?* — is a claim about what happens *across* frameworks. With $n = 1$ the curve is a shape, not an argument.

Target: CompuCell3D (Cellular Potts). Nominated by the jax-morph note before any of this work, on the grounds that it is *the biggest step away from everything we have*: a cell is not a point with per-cell state, it is a **set of lattice sites**. Its position is a derived centroid. Its dynamics are Metropolis flips of site ownership. There is no per-cell equation to subtract.

That is the point. Every measurement in the jax-morph atlas rested on cells being points and a differential test being a subtraction. **If the procedure is general it survives this; if it is secretly a particle-framework procedure, this is where it breaks.**

The prediction — made before running anything, and the reason this note exists — is in `TRANSFER.md`.

2. What transferred, measured rather than asserted

`STATUS.md` in the jax-morph atlas claimed ∼60% of the Python was instrument and would copy as-is. Forking it is the test of that claim, and it came out slightly better than claimed.

file	edits	why
`record.py`	none	schema, ladder and the twelve rules are about the record, not the target. Self-test passes unchanged.
`saturation.py`	none	to *transfer*: the ledger scores against the frozen baseline and nothing in it knows what a cell is. It was later *extended* — see §5 — but that is a defect the second repository exposed, not a porting cost.
`atlas.py`	none	blast radius, rung, commit-or-revert — all `HERE`-relative.
`registry_view.py`	none	reads `plexus.operators`. Produced the same 52-contract baseline from the new folder.
`agents/`	none	the six roles describe a job, not a repository.
`run_spec.py`	5 lines	output paths only (`log/atlas_cc3d/`, `graphs_data/atlas_cc3d/`).
`oracle.py`, `inventory.py`, `paper.py`	rewrite	target-specific by construction: the reference, its mechanism scan, its PDF.

Five of the seven instruments needed zero edits. That is the strongest evidence so far that the apparatus is a method rather than a one-off, and it cost about ten minutes to establish.

3. The oracle — where the risk actually is

`TRANSFER.md` predicted the oracle would be the largest risk, and a *target* risk rather than a *procedure* risk. It was both, and **it now passes**.

gate	state	
installs	**done**	CompuCell3D 4.10.0 from its own conda channel into an isolated environment (`/workspace/.conda_envs/cc3d-oracle`), py312.
imports	**done**	after supplying `libGL` — the container is headless and CompuCell3D links its graphics stack unconditionally.
model definable in Python	**done**	`cc3d.core.PyCoreSpecs`, so the model is written in Python and never hand-typed as XML.
isolation	**done**	the Plexus interpreter *cannot* `import cc3d`. Checked, not assumed: a process that can reach the reference can borrow its answer.
runs headless	**done**	9 cells at 30^2, 45 at 60^2; artefacts with provenance in `_oracle/runs/smoke/`.
deterministic at a fixed seed	**done**	two runs at seed 42 are *bit-identical* across id, type, volume, surface and centre of mass; seed 7 differs.

Getting there meant routing around three defects in CompuCell3D 4.10.0, none of them configuration:

1. The advertised pure-Python route (`PyCoreSpecs` → `service_cc3d`) passes the *list of specs* where a simulation file is expected: `get_custom_settings_path()` does `Path(<list>)`. Shimming that only exposes the next one — `CC3DSimService._run` asserts `os.path.isfile(<list>)`. **That route simply does not work in this release.**

2. CC3D `execs` the project's Python script with its own globals, so a steppable class defined in the same file raises `NameError: SteppableBasePy is not defined` at registration. The class must live in a separate importable module.
3. The output directory may not sit inside the project folder.

The way through is to use `PyCoreSpecs` for what it is genuinely good at — generating correct CC3DML, `<RandomSeed>` included — write a real `.cc3d` project, and run it through `cc3d/run_script.py`, the officially supported headless entry point, which never touches the broken path. The model stays defined in Python; only the transport is a file.

Why this mattered enough to spend the time on. jax-morph was chosen *because* it ran, and the first note said so in its own Phase 0. A target that cannot be run reduces the atlas to the Okuda situation — wrong operator, wrong parameter or wrong reading of a figure, with no way to tell which. **Determinism is the gate behind the gate**: without it, every later disagreement is unattributable between their noise and our error. Both now hold, so Phase 1 is unblocked.

4. The phases

Seven stages, the same ladder as the first atlas. **I stop at the end of each one and wait for you.** What differs is not the procedure but what each stage has to mean when a cell is a region of a lattice rather than a point.

Phase 0 — Instruments and the oracle DONE

Goal. Make every later measurement trustworthy before taking one.
The instruments came across for free (§2). **The oracle was the phase**, and it is where the risk always was: install, import, isolate, run headless, and be deterministic at a fixed seed. **All six gates now pass** (§3), including the stop condition — the authors' own cell-sorting model runs twice from one seed and is bit-identical. Until that held, a later disagreement would have been unattributable between their noise and our error. It holds.

Phase 1 — Read every mechanism not started

The excavator on each mechanism: what it does to the state, its equations, the role of every knob, and the `surprises` a reimplementer gets wrong.
What is different here. jax-morph's mechanisms were 20 classes plus 4 architectural contracts a scan could not see. CompuCell3D's are *plugins* and *steppables* declared through XML or `PyCoreSpecs` — a different extraction problem, and one where the scan-versus-hand split will fall in a different place. **The 4 hand-added entries were among the most interesting things the first atlas found**; expecting an automatic scan to be sufficient here would be repeating a mistake we already know about.

Phase 2 — Normalize, and let the skeptic attack — not started

Each mechanism becomes a typed Plexus contract with a verdict, then the skeptic tries to break it.

The prediction is already on the record (`TRANSFER.md`): the *biological* mechanisms — growth, division, death, chemotaxis, secretion, diffusion — should come back largely `alias` or `implementation` of contracts the first atlas already registered, while the *representational* ones — cell-as-domain, the Metropolis acceptance rule, contact energy over shared boundary, volume and surface constraints — should be genuinely new. If that splits as predicted, the conclusion is that *the biological vocabulary is converging while the representational vocabulary is not.*

Phase 3 — Implement what is missing — not started

Every `new` and every `refinement` becomes a Plexus operator in the anti-chamber, with a test of one property statable without the reference: a conservation law, a sign, a limit, a symmetry.

The open question this phase will force. Plexus's `cell` set is points with per-cell state. A Potts cell is a *set of lattice sites*. Either that is expressible as an existing set with a different map, or the language needs a representation it does not have — and finding out which is worth more than any individual operator here.

Phase 4 — Differential validation — not started

Metric and threshold written down *first*; then the same initial condition through both implementations; then subtract.

This is the phase most likely to break, and the reason this target was chosen. Every differential test in the first atlas was a subtraction of per-cell arrays. Here there is no per-cell equation to compare and the dynamics are a Metropolis chain, so a matched trajectory is not even meaningful — the same argument that made the division differ compare a pooled *hazard* instead of cell counts, but now applying to the whole framework rather than one operator. The tests will have to be **distributional**: equilibrium sorted-boundary length, volume distribution, contact-energy spectrum.

Phase 5 — The forward figure — not started

One headline result of the paper reproduced inside Plexus out of normalized operators, and set *beside* the reference rather than described next to it. Cell sorting is the obvious candidate: it is CompuCell3D's canonical demonstration and its outcome is a topology, not a trajectory.

Phase 6 — Differentiability — not started

Probably not applicable, and saying why is the result. The first atlas's Phase 6 rested on the forward model being differentiable. A Metropolis acceptance is a discrete accept/reject on an energy difference — there is no pathwise gradient through it at all, and the straight-through trick that carried the division draw does not obviously transfer. If Plexus can express Potts dynamics forward but not inversely, that is a real and reportable limit of the language.

Phase 7 — Promotion, and the joint ledger — not started

The measurement only becomes an argument here: the cumulative-new curve across *two* repositories. Flat means the language is saturating; still climbing means it is not.

5. An instrument defect only the second repository could reveal — fixed

`saturation.py` classifies a mechanism `implementation` when it reaches a contract *this run* has already counted as new. At $n = 1$ that is complete. At $n = 2$ it is wrong: every contract the first atlas introduced would be counted as new *again* by the second, inflating precisely the number the instrument exists to protect.

The fix is a `--prior` flag: an earlier `atlas_record.yaml` pre-seeds the "already spoken for" set, so a second sighting reads as an implementation *of that repository* rather than as new vocabulary. Checked by feeding jax-morph's record back to itself — its 8 new contracts all demote and it scores **NEW 0, implementation 14**, which is the correct answer for a repository measured against itself.

Applied to *both* campaigns' copies, so the instrument stays one instrument. That is the whole value of a second target arriving before the first is promoted: **the defect was invisible at $n = 1$ and structural at $n = 2$**, and it would have silently doubled the headline number.

Where things are. `TRANSFER.md` — the prediction, written first. `../atlas_jax_morph/STATUS.md` — what the first atlas produced and what was reusable. `_state/registry_baseline.json` — the same frozen 52 contracts, so the two measurements are comparable.